the task from the queue ❺, releases the lock on the queue, and then runs the task ❻. The future associated with the task will then be made ready when the task completes.

Posting a task on the queue is equally simple: a new packaged task is created from the supplied function ❼, the future is obtained from that task ❽ by calling the `get_future()` member function, and the task is put on the list ❾ before the future is returned to the caller ❿). The code that posted the message to the GUI thread can then wait for the future if it needs to know that the task has been completed, or it can discard the future if it doesn't need to know.

This example uses `std::packaged_task<void()>` for the tasks, which wraps a function or other callable object that takes no parameters and returns `void` (if it returns anything else, the return value is discarded). This is the simplest possible task, but as you saw earlier, `std::packaged_task` can also be used in more complex situations—by specifying a different function signature as the template parameter, you can change the return type (and thus the type of data stored in the future's associated state) and also the argument types of the function call operator. This example could easily be extended to allow for tasks that are to be run on the GUI thread to accept arguments and return a value in the `std::future` rather than just a completion indicator.

What about those tasks that can't be expressed as a simple function call or those tasks where the result may come from more than one place? These cases are dealt with by the third way of creating a future: using a `std::promise` to set the value explicitly.

4.2.3. Making (std::)promises

When you have an application that needs to handle a lot of network connections, it's often tempting to handle each connection on a separate thread, because this can make the network communication easier to think about and easier to program. This works well for low numbers of connections (and thus low numbers of threads). Unfortunately, as the number of connections rises, this becomes less suitable; the large numbers of threads consequently consume large numbers of operating system resources and potentially cause a lot of context switching (when the number of threads exceeds the available hardware concurrency), impacting performance. In the extreme case, the operating system may run out of resources for running new threads before its capacity for network connections is exhausted. In applications with very large numbers of network connections, it's therefore common to have a small number of threads (possibly only one) handling the connections, each thread dealing with multiple connections at once.

Consider one of these threads handling the connections. Data packets will come in from the various connections being handled in essentially random order, and likewise data packets will be queued to be sent in random order. In many cases, other parts of the application will be waiting either for data to be successfully sent or for a new batch of data to be successfully received via a specific network connection.

`std::promise<T>` provides a means of setting a value (of type `T`), which can later be read through an associated `std::future<T>` object. A `std::promise`/`std::future` pair would provide one possible mechanism for this facility; the waiting thread could block on the future, while the thread providing the data could use the promise half of the pairing to set the associated value and make the future *ready*.